

Full Stack Development (FSD)
Documentation

Book A Doctor

A MERN Stack Doctor Appointment Booking System

Phase: Project Documentation

Document Version: 1.0

Date: 18th of June, 2026

Prepared by: Chethula Harish

Project Type: Individual Project

Classification: Project Documentation — For Academic Submission

Table of Contents

1. Introduction.....	3
2. Project Overview.....	4
3. System Architecture.....	6
4. Database Design.....	8
5. UML Class Diagram.....	11
6. Setup Instructions.....	14
7. Folder Structure.....	15
8. Running the Application.....	16
9. API Documentation.....	17
10. Authentication & Authorization.....	18
11. User Interface.....	19
12. Testing.....	22
13. Screenshots / Demo.....	24
14. Known Issues.....	26
15. Future Enhancements.....	27
16. Conclusion.....	28

1. Introduction

1.1 Project Title

Book A Doctor — A Production-Grade MERN Stack Doctor Appointment Booking System.

1.2 Team Members

Chethula Harish — Lead Full-Stack Developer & Solutions Architect

- Role: End-to-end implementation of backend REST API, database schema modelling, frontend single-page application development, authentication and authorization orchestration, role-based dashboard design, and deployment across cloud platforms.

1.3 Purpose of the Document

This document presents a comprehensive technical and functional overview of the Book A Doctor application, developed as part of the VIP C2 Programme on Full Stack Development with the MERN stack. It is intended to serve as a complete academic record of the system's architecture, design decisions, implementation details, and verification process, and is submitted in fulfilment of the project documentation requirement of the course.

1.4 Scope

The scope of this document covers the complete software development lifecycle of the application — from requirement analysis and system design through to implementation, testing, and deployment. It includes the database schema, UML class structure, REST API contract, authentication mechanism, user-interface walkthrough, testing strategy, and a discussion of known limitations and proposed future enhancements.

2. Project Overview

2.1 Purpose

Book A Doctor is developed to streamline the process of medical consultation scheduling between patients and verified medical practitioners. Traditional appointment booking workflows in clinics and hospitals are often manual, time-consuming, and prone to scheduling conflicts. This platform digitises the entire workflow — from doctor on-boarding and administrative verification, to appointment booking and status tracking — within a single, secure, role-aware web application.

For patients, the system offers a simple interface to browse a directory of approved doctors and book consultations. For medical practitioners, it offers a dedicated portal to manage incoming appointment requests. For platform administrators, it provides full oversight of doctor on-boarding and user governance.

2.2 Problem Statement

Patients frequently face difficulty in identifying available doctors by specialisation and securing timely appointments, while clinics lack a low-cost digital mechanism to manage practitioner credentialing and appointment scheduling. Book A Doctor addresses this gap by providing a unified portal with distinct, permission-scoped views for each class of user.

2.3 Key Features

User Module

- Account Registration — secure sign-up with hashed credential storage.
- Login / Authentication — JWT-based session issuance on successful login.
- Browse Doctors — view the directory of admin-approved medical practitioners.
- Book Appointments — schedule a consultation with a doctor of choice by date and time.
- View Notifications — receive real-time status updates regarding appointment requests and doctor application outcomes.

Doctor Module

- Apply as Doctor — any registered user may submit a doctor application with professional details (name, specialisation, experience, contact information).
- Manage Appointments — review all appointments booked against the doctor's profile.
- Update Appointment Status — approve, reject, or mark appointments as completed.

Admin Module

- Approve / Reject Doctor Applications — review pending applications and grant or deny practitioner status.
- Manage Users — administrative oversight of all registered platform accounts.

- Platform Administration — centralised control over doctor records and platform-wide data integrity.

2.4 Target Users

User Role	Description
Patient (User)	Registers, searches the doctor directory, and books appointments.
Doctor	A user account elevated to practitioner status after admin approval; manages incoming appointments.
Administrator	Verifies doctor credentials and oversees platform-wide governance.

3. System Architecture

3.1 Architectural Style

Book A Doctor follows a decoupled three-tier client-server architecture, consistent with conventional MERN stack design. The presentation layer (React.js SPA), the application/business logic layer (Node.js with Express.js REST API), and the data persistence layer (MongoDB Atlas) are deployed and scaled independently, communicating exclusively over stateless HTTPS REST/JSON requests.

3.2 Frontend Architecture

The client is implemented as a single-page application (SPA) using React.js. Core architectural elements include:

- **Redux:** Centralised, predictable global state management for authenticated user sessions, role flags (isDoctor / isAdmin), and shared UI state across components.
- **React Router:** Declarative, client-side route management enabling role-scoped navigation between the Dashboard, Doctors, Apply Doctor, Appointments, Notifications, and Admin views without full-page reloads.
- **Axios:** A promisified HTTP client used for all REST API communication, configured to attach authentication tokens and centralise error handling for failed requests.
- **Responsive Component Design:** A reusable component hierarchy (sidebar navigation, dashboard cards, data tables, and forms) styled for desktop and mobile viewports.

3.3 Backend Architecture

The server runtime is powered by Node.js and structured using Express.js, following a layered Model-Controller-Route architecture for maintainability and separation of concerns:

- **Routing:** Modular, decoupled Express Router endpoints grouped by domain — authentication, doctor management, appointment scheduling, user notifications, and administrative actions.
- **Controllers:** Encapsulate business logic for each route, including validation, status transitions, and database interaction via Mongoose models.
- **Middleware:** Dedicated middleware functions intercept incoming requests for JWT verification (protect), role-based authorisation checks (isAdmin, isDoctor), and centralised error handling.

3.4 Database Layer

MongoDB Atlas serves as the cloud-hosted NoSQL data store, accessed and modelled through the Mongoose Object Data Modelling (ODM) library. Mongoose enforces schema-level data integrity, type validation, and referential linkage between collections using ObjectId references — connecting users, doctor applications, appointments, and notifications.

3.5 Deployment Topology

The application follows a decoupled cloud deployment model. The React frontend is statically built and deployed on Vercel, providing global CDN-backed delivery and automatic CI/CD on every code push. The Node.js/Express backend, along with its REST API surface, is deployed independently, while MongoDB Atlas hosts the persistent data layer as a fully managed cloud database cluster.

System Architecture Flow

The system architecture can be summarised as a layered pipeline: the React.js SPA (Frontend, hosted on Vercel) communicates over HTTPS REST/JSON with the Express.js Router layer (Backend), which is gated by JWT authentication and role-based middleware before reaching the Controllers (business logic). The Controllers interact with MongoDB Atlas Collections via the Mongoose ODM, which enforces schema validation prior to any read or write operation. Below is a structured representation of this flow.



Figure 3.1 — Full System Architecture Flow Diagram

4. Database Design

4.1 Overview

The data layer is composed of four primary MongoDB collections — User, Doctor, Appointment, and Notification — modelled using Mongoose schemas. Referential relationships between collections are implemented through ObjectId references rather than embedded sub-documents, favouring normalisation and reducing data duplication.

4.2 Entity Definitions

4.2.1 USER

Field	Type	Constraint	Description
_id	ObjectId	PK	Unique identifier for the user document.
name	String	Required	Full name of the registered user.
email	String	Unique, Required	Login identifier; used for authentication.
password	String	Required	BCrypt-hashed password credential.
isDoctor	Boolean	Default: false	Flag indicating approved practitioner status.
isAdmin	Boolean	Default: false	Flag indicating administrative privileges.
createdAt	Date	Auto-generated	Timestamp of account creation.

4.2.2 DOCTOR

Field	Type	Constraint	Description
_id	ObjectId	PK	Unique identifier for the doctor application record.
userId	ObjectId	FK → User	Reference to the originating user account.
firstName	String	Required	Practitioner's first name.
lastName	String	Required	Practitioner's last name.
phone	String	Required	Contact phone number.
specialization	String	Required	Medical field of expertise (e.g., Neurology, Cardiology).
experience	Number	Required	Years of professional clinical experience.
status	String	pending / approved /	Current state of the application as reviewed by an administrator.

Field	Type	Constraint	Description
		rejected	

4.2.3 APPOINTMENT

Field	Type	Constraint	Description
_id	ObjectId	PK	Unique identifier for the appointment record.
userId	ObjectId	FK → User	The patient who booked the appointment.
doctorId	ObjectId	FK → Doctor	The doctor assigned to the appointment.
date	Date	Required	Scheduled date of the consultation.
time	String	Required	Scheduled time slot of the consultation.
status	String	pending / approved / completed / cancelled	Current lifecycle state of the appointment.
createdAt	Date	Auto-generated	Timestamp of appointment creation.

4.2.4 NOTIFICATION

Field	Type	Constraint	Description
_id	ObjectId	PK	Unique identifier for the notification record.
userId	ObjectId	FK → User	Recipient of the notification.
message	String	Required	Notification content/text.
read	Boolean	Default: false	Read/unread state of the notification.
createdAt	Date	Auto-generated	Timestamp of notification creation.

4.3 Relationships

- One User can book many Appointments (1 : N).
- One Doctor can manage/attend many Appointments (1 : N).
- One User can receive many Notifications (1 : N).
- One User can apply as exactly one Doctor profile per application (1 : 1, repeatable per submission).

4.4 Entity-Relationship Diagram

The diagram below illustrates the four core collections and the directional relationships between them, as implemented in the Mongoose schema layer. The User collection acts as the central entity, linking outward to Doctor (applies as), Appointment (books), and Notification (receives), while the Doctor collection links to Appointment (attends).

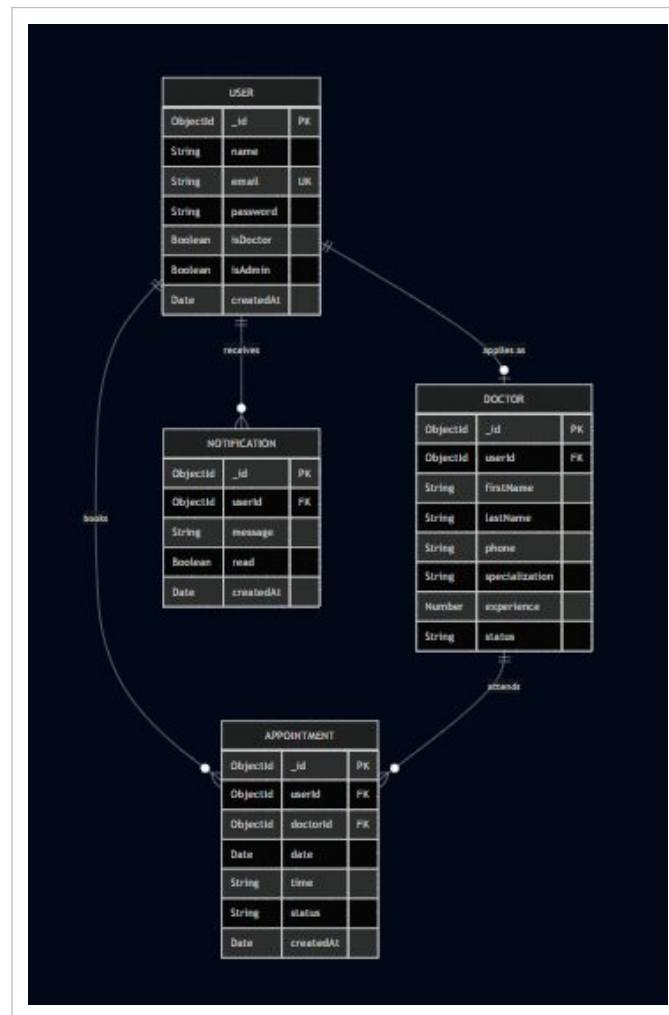


Figure 4.1 — Entity-Relationship Diagram — User, Doctor, Appointment, and Notification Collections

5. UML Class Diagram

5.1 Class Overview

The system is modelled around five primary classes — User, Doctor, Appointment, Notification, and Admin — each encapsulating both its data attributes and the operations it exposes. The class diagram captures both the static structure of the domain model and the principal interactions between actors and the system.

5.2 Class Responsibilities

User

Attributes: ObjectId _id, String name, String email, String password, Boolean isDoctor, Boolean isAdmin

Methods:

- register() — creates a new user account with hashed credentials.
- login() — authenticates a user and issues a signed JWT.
- bookAppointment() — creates a new appointment request against a chosen doctor.
- viewNotifications() — retrieves all notifications addressed to the user.
- logout() — invalidates the active client-side session.

Doctor

Attributes: ObjectId _id, ObjectId userId, String firstName, String lastName, String specialization, Number experience, String status

Methods:

- applyDoctor() — submits a new practitioner application for admin review.
- viewAppointments() — lists all appointments assigned to the doctor.
- updateAppointmentStatus() — transitions an appointment's status (approve / reject / complete).

Appointment

Attributes: ObjectId _id, ObjectId userId, ObjectId doctorId, Date date, String time, String status

Methods:

- createAppointment() — instantiates a new appointment booking record.
- updateStatus() — modifies the appointment's lifecycle state.
- cancelAppointment() — withdraws an active appointment request.

Notification

Attributes: ObjectId _id, ObjectId userId, String message, Boolean read

Methods:

- createNotification() — generates a new notification entry for a user.
- markAsRead() — updates the read flag once the user views the notification.

Admin

Methods:

- approveDoctor() — grants practitioner status to a pending doctor application.
- rejectDoctor() — denies a pending doctor application.
- manageUsers() — administrative oversight of registered user accounts.
- manageDoctors() — administrative oversight of all doctor records.

5.3 UML Class Diagram

The diagram below presents the static class structure with the cardinality of each relationship — an Admin moderates many Doctor records (1 : N); a User books and manages many Appointments (1 : N); a Doctor attends and reviews many Appointments (1 : N); and a User receives many Notifications (1 : N).

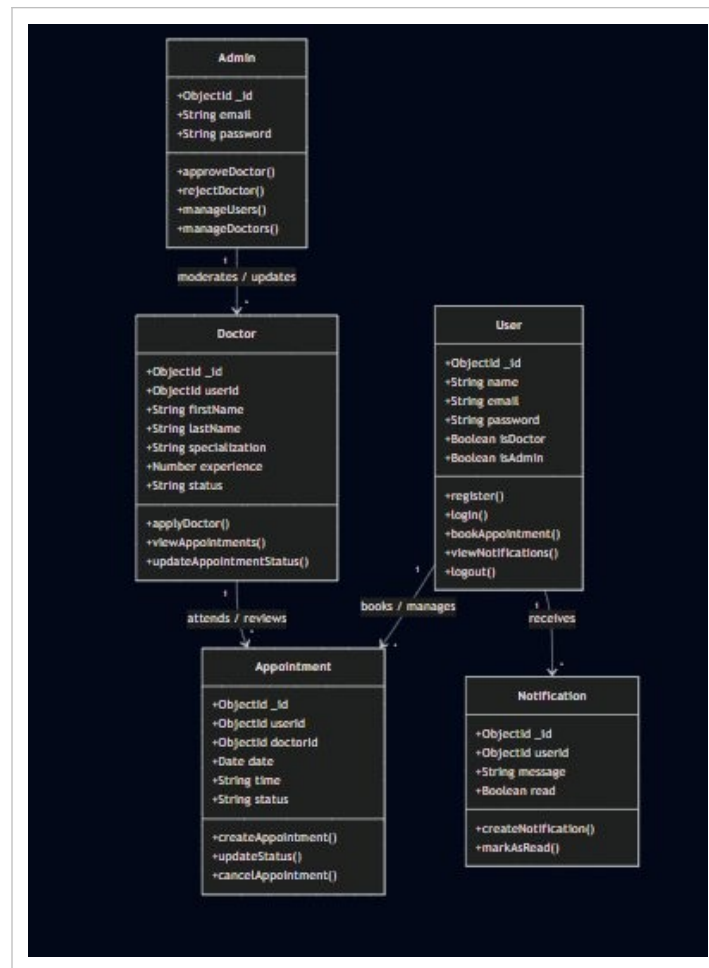


Figure 5.1 — UML Class Diagram — User, Doctor, Appointment, Notification, and Admin Classes

6. Setup Instructions

6.1 Prerequisites

- Node.js: v18 LTS or above installed.
- MongoDB: A cloud cluster on MongoDB Atlas (recommended) or a local installation (v6.x or above).
- Package Manager: npm v9.x or above.
- Code Editor: Visual Studio Code (recommended).
- Version Control: Git, with a GitHub account for repository hosting and Vercel deployment integration.

6.2 Installation Steps

Step 1 — Clone the Repository

```
git clone <repository-url>
cd book-a-doctor
```

Step 2 — Configure Environment Variables

Create a .env file under the /server (or /backend) directory with the following variables:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/
bookadoctor
JWT_SECRET=your_jwt_secret_key
```

Step 3 — Install Dependencies

```
# Install server dependencies
cd server
npm install

# Install client dependencies
cd ../client
npm install
```

Step 4 — Start the Application

Refer to Section 8 (Running the Application) for the complete startup sequence.

7. Folder Structure

7.1 Client

```
client/
├─ public/           # Static assets
└─ src/
  ├─ assets/         # Global CSS and themes
  ├─ components/     # Reusable presentation parts (Sidebar, Navbar,
Cards)
  │   ├─ redux/      # Store configuration, slices/reducers, actions
  │   ├─ pages/      # View routes (Login, Register, Dashboard,
  │   │             # Doctors, ApplyDoctor, Appointments,
  │   │             # Notifications, AdminDoctors)
  │   └─ App.jsx     # React Router setup
  └─ main.jsx        # DOM mounting entry point
```

7.2 Server

```
server/
├─ config/           # db.js (Mongoose connection pool setup)
├─ controllers/      # Auth, doctor, appointment, notification, admin
controllers
├─ middleware/       # authMiddleware.js & errorMiddleware.js
├─ models/           # Mongoose schema definitions (User, Doctor,
  │                 # Appointment, Notification)
├─ routes/           # Express REST endpoint routes
└─ server.js         # Application entry file
```

8. Running the Application

To run the application locally in development mode, both the backend API server and the frontend development server must be started concurrently in separate terminal sessions.

8.1 Backend Server

Run the following command in the /server directory:

```
npm start  
  
# Default port: http://localhost:5000
```

8.2 Frontend Server

Run the following command in the /client directory:

```
npm start  
  
# Default port: http://localhost:3000
```

8.3 Production Deployment

In production, the frontend is built as a static bundle and deployed to Vercel, where it is automatically rebuilt and redeployed on every push to the main branch. The backend Express API is hosted as an independent service, with MongoDB Atlas serving as the persistent cloud data store accessible to the deployed backend over a secured connection string.

9. API Documentation

9.1 Authentication Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user account.
POST	/api/auth/login	No	Authenticate a user and issue a JWT.

9.2 Doctor Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/doctor/apply	Yes (User)	Submit a new doctor application for admin review.
GET	/api/doctor/getAllDoctors	Yes	Retrieve the list of all admin-approved doctors.

9.3 Appointment Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/appointment/book	Yes (User)	Book a new appointment with a selected doctor.

9.4 User & Notification Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/user/getNotifications	Yes	Retrieve all notifications addressed to the authenticated user.

9.5 Administrative Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/admin/approveDoctor	Yes (Admin)	Approve a pending doctor application, elevating the corresponding user account to practitioner status.

10. Authentication & Authorization

10.1 Authentication Mechanism

Authentication in Book A Doctor is implemented using JSON Web Tokens (JWT), providing a stateless mechanism for verifying user identity on each request without server-side session storage.

- **Password Hashing:** User passwords are never stored in plaintext. The `bcryptjs` library is used to generate a salted hash of each password prior to persistence in the database, and to securely compare hashes during login.
- **Token Issuance:** Upon successful registration or login, the server generates a signed JWT containing the user's identifier and role flags, using a secret key (`JWT_SECRET`).
- **Token Verification:** Protected API routes validate the JWT on every incoming request before granting access to the underlying controller logic.

10.2 Authorization & Role-Based Access Control

Beyond authentication, the platform enforces role-based access control (RBAC) to restrict functionality according to the user's role — User, Doctor, or Admin:

- **Protected Routes:** Middleware intercepts requests to sensitive endpoints and rejects any request lacking a valid token.
- **Role Verification:** Dedicated middleware checks the `isDoctor` and `isAdmin` flags on the authenticated user's record before allowing access to doctor-only or admin-only operations (e.g., approving a doctor application, updating appointment status).
- **Dashboard Scoping:** The frontend renders distinct navigation menus and dashboard views depending on the authenticated role — Dashboard, Doctors, Apply Doctor, Appointments, and Notifications for standard Users; Dashboard and Appointments for Doctors; and Dashboard and Doctors (request management) for Admins.

10.3 Illustrative Middleware Logic

```
const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Not authorized' });
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
};
```

11. User Interface

The following screenshots showcase the principal interface views of the Book A Doctor application, captured from the deployed Vercel build, illustrating the workflow across the User, Doctor, and Admin roles.

11.1 Login Interface

Figure 11.1 below presents the application's login screen. The interface requires a registered email address and password, and provides a link for new users to navigate to the registration screen.

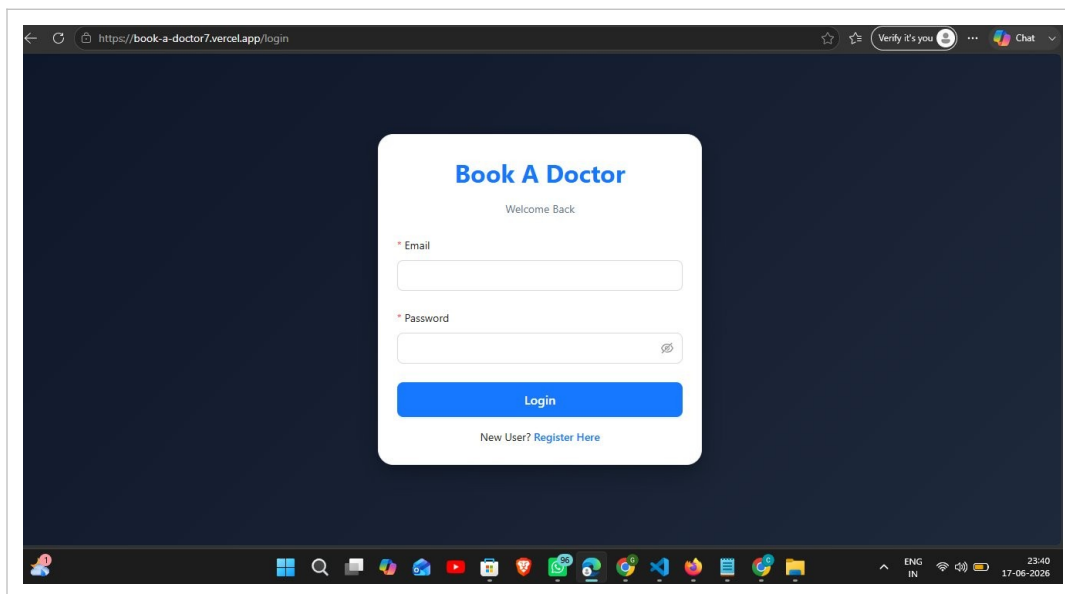


Figure 11.1 — Login Interface — Book A Doctor Application

11.2 User Dashboard

Figure 11.2 presents the dashboard view rendered for an authenticated standard user. It displays summary metrics — total available doctors, the user's appointment count, and unread notification count — alongside a sidebar offering navigation to Doctors, Apply Doctor, Appointments, and Notifications.

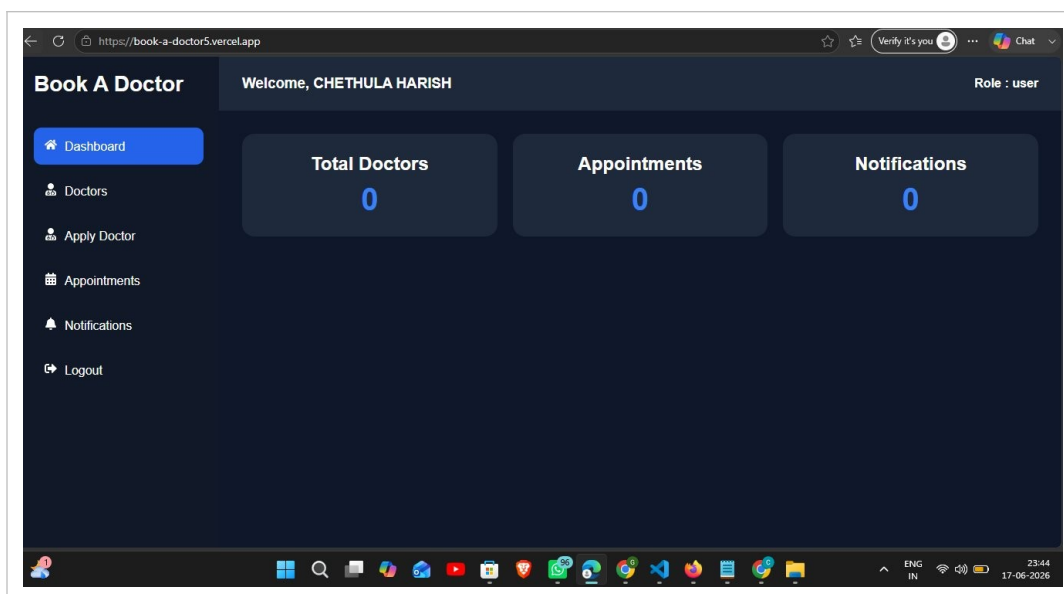


Figure 11.2 — User Dashboard — Summary Metrics and Sidebar Navigation

11.3 Apply As Doctor Form

Figure 11.3 shows the Apply As Doctor form, through which a registered user submits professional credentials — first name, last name, phone number, email, specialisation, and years of experience — for administrative review and approval.

A screenshot of the "Apply As Doctor" form within the "Book A Doctor" application. The user is "CHETHULA HARISH" with the role of "user". The form is titled "Apply As Doctor" and contains several input fields with red asterisks indicating required fields. The fields are: "First Name" (filled with "harish"), "Last Name" (filled with "yadav"), "Phone" (filled with "6305421986"), "Email" (filled with "harishchethula892@gmail.com"), "Specialization" (filled with "neuro"), and "Experience" (empty). The sidebar on the left is the same as in Figure 11.2, with "Apply Doctor" selected. The browser's address bar shows "https://book-a-doctor5.vercel.app". The Windows taskbar at the bottom shows the system clock as 23:46 on 17-06-2026.

Figure 11.3 — Apply As Doctor — Professional Credential Submission Form

11.4 Doctor Dashboard

Once a doctor application is approved by an administrator, the user's role is elevated and the dashboard is updated accordingly, as shown in Figure 11.4. The doctor-specific dashboard displays

the total number of registered doctors on the platform, the doctor's own appointment count, and any pending notifications, alongside a reduced sidebar scoped to Dashboard and Appointments.

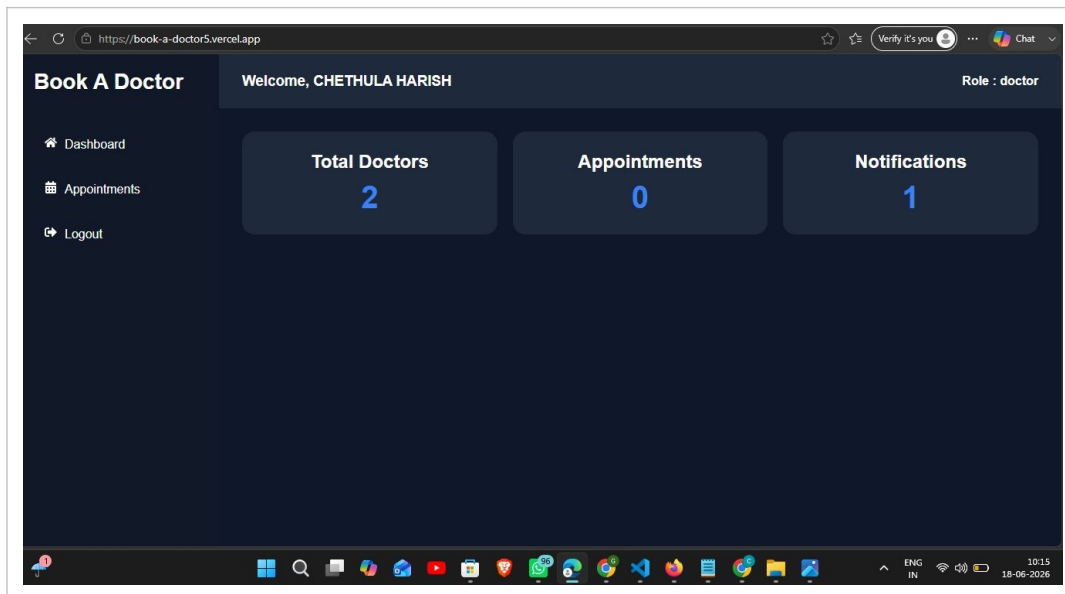


Figure 11.4 — Doctor Dashboard — Role-Elevated View After Admin Approval

11.5 Admin Doctor Approval Panel

Figure 11.5 illustrates the administrative Doctor Requests panel, where an Admin reviews all submitted doctor applications, including the applicant's name, declared specialisation, years of experience, and current status. Pending applications display an Approve action button, while previously approved applications are clearly marked.

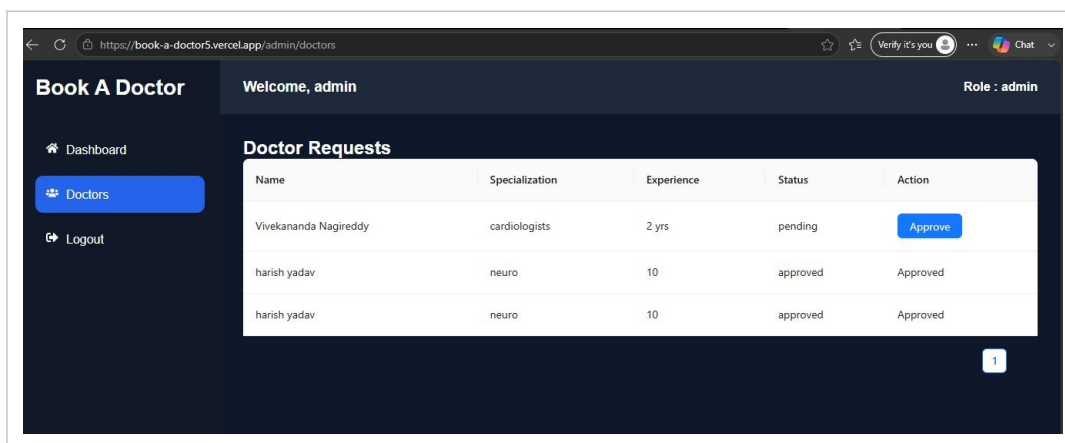


Figure 11.5 — Admin Panel — Doctor Requests and Approval Workflow

12. Testing

The testing strategy for Book A Doctor ensures functional correctness, data integrity, and security enforcement across all three user roles, validated at multiple levels of granularity.

12.1 Unit Testing

Unit-level verification was performed on isolated utility functions and validation logic, including:

- Password hashing and comparison routines (bcryptjs).
- JWT token generation and verification logic.
- Form-level input validation rules on the React client (required fields, email format, numeric experience values).

12.2 Integration Testing

API-level integration testing was conducted using Postman to verify request/response correctness, authentication enforcement, and status code accuracy across endpoints, including:

- Verifying that protected routes correctly reject requests lacking a valid JWT (401 Unauthorized).
- Verifying that admin-only routes reject requests from non-administrative accounts (403 Forbidden).
- Verifying correct state transitions on doctor application status (pending → approved) and on appointment status (pending → approved/completed).

12.3 User Acceptance Testing (UAT)

A series of manual UAT scripts were executed to validate the complete end-to-end user journey across all three roles, covering the flow: Registration → Login → Browse Doctors → Apply as Doctor → Admin Approval → Book Appointment → Doctor Status Update → Notification Receipt.

Test Case ID	Scenario	Expected Result	Status
UAT-01	New user registration with valid credentials	Account created; redirected to login	Pass
UAT-02	Login with valid credentials	JWT issued; redirected to role-based dashboard	Pass
UAT-03	Login with invalid credentials	Error message displayed; access denied	Pass
UAT-04	Submit Apply Doctor form with complete details	Application recorded with status 'pending'	Pass
UAT-05	Admin approves a pending doctor application	User role updated to doctor; dashboard reflects new role	Pass

Test Case ID	Scenario	Expected Result	Status
UAT-06	Patient books an appointment with an approved doctor	Appointment created with status 'pending'	Pass
UAT-07	Doctor updates appointment status	Status change reflected on patient's dashboard	Pass
UAT-08	Unauthenticated access to a protected route	Request rejected with 401 status	Pass

13. Screenshots / Demo

The platform was developed and deployed in a decoupled cloud environment using Vercel for hosting. The figures in this section document the live deployment workflow and the deployed application URLs used for the academic demonstration of this project.

13.1 Deployment Resources

Resource	URL / Details
Live Application	https://book-a-doctor5.vercel.app/
Deployment Platform	Vercel (Frontend) — Auto CI/CD on push to main branch
Database	MongoDB Atlas (Cloud-hosted cluster)
Sample User Role	Role : user — CHETHULA HARISH
Sample Doctor Role	Role : doctor — CHETHULA HARISH (post-approval)
Sample Admin Role	Role : admin

13.2 Deployment Confirmation

Figure 13.1 below confirms successful deployment of the Book A Doctor frontend to Vercel's hosting infrastructure, with the application's login screen rendering correctly immediately following deployment.

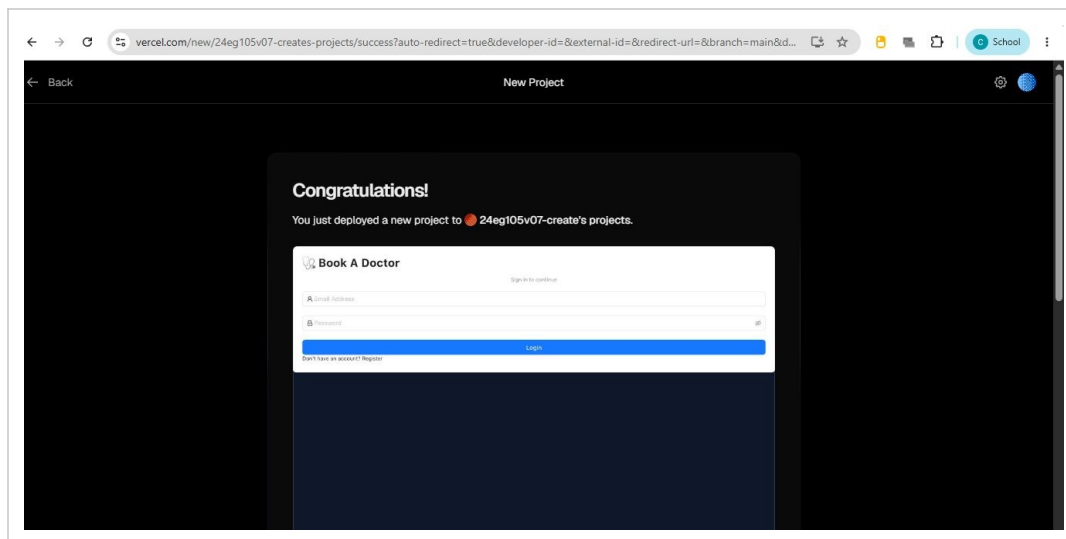


Figure 13.1 — Successful Deployment Confirmation on Vercel

13.3 Vercel Project Dashboard

Figure 13.2 presents the Vercel project management dashboard, confirming the live project status, deployment history, and resource usage metrics for the hosted application.

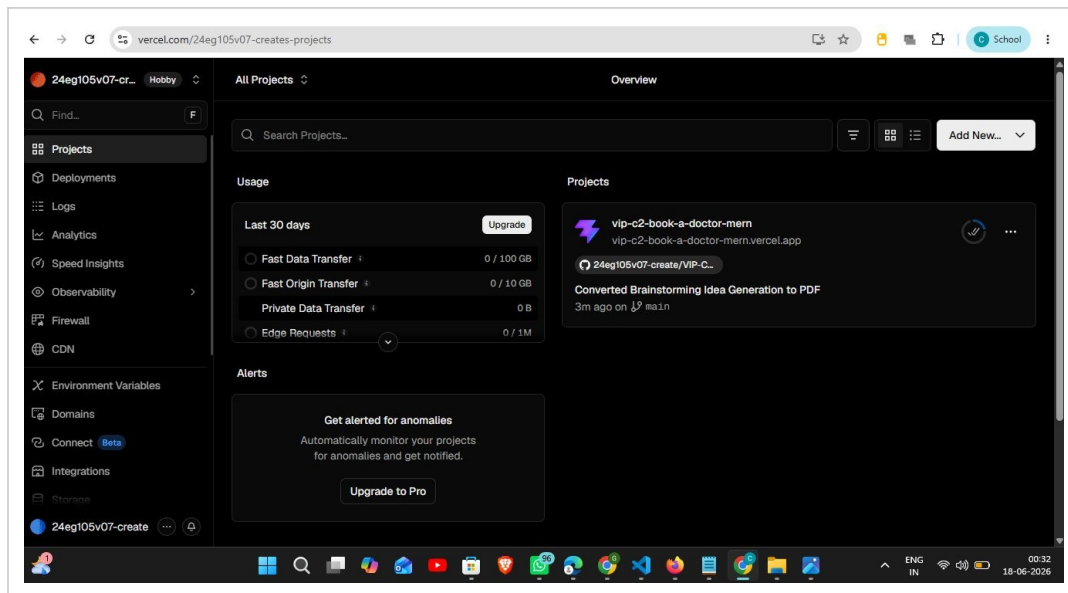


Figure 13.2 — Vercel Project Dashboard — Live Deployment and Resource Usage

14. Known Issues

Issue ID	Description	Mitigation / Plan
KI-01	Free-tier hosting on the backend service introduces a cold-start delay of approximately 30–60 seconds after periods of inactivity.	Upgrade to a paid hosting tier for production deployment to maintain a persistently warm server instance.
KI-02	The platform does not currently send email notifications for appointment confirmations, status changes, or doctor approval outcomes.	Integrate a transactional email service (e.g., Nodemailer with an SMTP provider) in a future release.
KI-03	The system does not currently validate appointment time slots against a doctor's existing bookings, which may permit overlapping or conflicting appointment requests.	Introduce server-side conflict validation logic that checks for overlapping date/time slots prior to confirming a new appointment.

15. Future Enhancements

Priority	Enhancement	Description
High	Online Payments	Integration with a payment gateway (e.g., Stripe/Razorpay) to support paid consultation bookings.
High	Email Notifications	Nodemailer-based transactional emails for appointment confirmations and status changes.
Medium	Video Consultation	In-browser video calling integration to support remote telemedicine consultations.
Medium	Doctor Ratings	Patient-submitted ratings and reviews for completed consultations, displayed on doctor profiles.
Future	AI Health Assistant	A conversational assistant to help patients triage symptoms and identify the appropriate specialisation.
Future	Mobile Application	A native or React Native companion application consuming the existing REST API.

16. Conclusion

Book A Doctor successfully demonstrates the application of the MERN stack (MongoDB, Express.js, React.js, and Node.js) to a real-world healthcare scheduling problem. The system implements a complete, role-aware workflow spanning patient registration, doctor on-boarding and administrative verification, and appointment lifecycle management, secured through JWT-based authentication, bcryptjs password hashing, and role-based access control.

Through this project, the complete software development lifecycle was undertaken individually — from requirement analysis and entity-relationship modelling, through REST API design and implementation, to frontend state management with Redux, and finally to cloud deployment and verification via manual and integration testing. The resulting platform validates the practicality of the MERN stack in delivering secure, scalable, and maintainable full-stack web applications.

The known limitations identified — namely hosting cold-start latency, the absence of email notifications, and the lack of appointment conflict validation — represent well-scoped opportunities for iterative improvement, alongside the broader set of future enhancements outlined in Section 15, including online payments, video consultations, and AI-assisted features. Taken together, this project serves as a comprehensive demonstration of full-stack development competency acquired through the VIP C2 Programme.